

Multi-Source Candidate Data Transformer

Technical Design — Eightfold Engineering Intern Assignment (Jul–Dec 2026)
Akshit Kalra · akshit0273.be23@chitkara.edu.in · Python 3.11

Problem. Candidate data arrives from many sources, for the same person, in conflicting formats. Produce **one canonical profile** per candidate — normalized, deduplicated, with **provenance** (where each value came from) and **confidence** (how sure we are). Guiding rule: **wrong-but-confident is worse than honestly-empty** — unknowns become `null`, never invented.

1 · PIPELINE



Key architectural choice: the internal **CanonicalRecord** is always complete and consistent; the **projection layer** is the *only* thing the runtime config touches — so config changes never require engine changes. **6 source adapters:** Recruiter CSV, ATS JSON (structured); GitHub API, Resume, LinkedIn, Recruiter notes (unstructured). Each is isolated & optional.

2 · CANONICAL SCHEMA & NORMALIZED FORMATS

FIELD	SHAPE / NORMALIZATION
candidate_id	sha1 of strongest identity key
full_name	string, whitespace-collapsed
emails / phones	string[]; phones → E.164
location	{city, region, country = ISO-3166 α2 }
links	{linkedin, github, portfolio, other[]}
years_experience	number null
skills	{[name, confidence, sources[]]; canonical names }
experience	{[company,title,start,end,summary]; dates YYYY-MM }
education	{[institution,degree,field,end_year]; YYYY }
provenance	{[field, source, method]}
overall_confidence	number 0–1

4 · RUNTIME CONFIGURABLE OUTPUT

A JSON config reshapes output — **same engine, no code changes**. It can: **select** a subset of fields · **rename / remap** via a `from path (emails[0], skills[].name) · set per-field normalization` (E164, canonical, lower/upper) · **toggle** provenance & confidence · choose **on_missing** = `null` | `omit` | `error`.

```
{ "path": "phone", "from": "phones[0]", "normalize": "E164", "required": true }
```

Projection resolves paths (dotted, `[i]` index, `[],field` list-projection) against the canonical record; then **validation** checks the result against the requested schema (types + required) before returning. Clean separation: internal record vs projection.

3 · MERGE, CONFLICT RESOLUTION & CONFIDENCE

Match keys (priority): normalized email → E.164 phone → github/linkedin URL → (last resort) exact name+company, *only* for records with no strong key. Records clustered via **union-find** → one profile per cluster.

Pick a winner: **list** fields (emails, phones, skills, links) → **union + dedupe**; **scalar** fields → highest **source trust**, tie-broken by **agreement count** → completeness → lexical (so input order never changes output).

Trust: `ATS .95` `CSV .90` `LinkedIn .75` `Resume .70` `Notes .60` `GitHub .55`

```
field_conf = trust(src) × agreement × completeness (clamped 0–1)
```

overall = weighted mean over populated fields. Every value carries **provenance** with method `←`
direct-normalized-inferred-merged → fully explainable.

5 · EDGE CASES & DELIBERATE SCOPE

- **Same person, conflicting names** (Bob ↔ Robert) → merge via shared email; higher-trust name wins, both kept in provenance.
- **Same name, different people** → emails differ ⇒ **not** merged; two profiles.
- **Garbage / missing source** (bad JSON, 404, missing file) → warn & skip; run never crashes.
- **Phone-format chaos** → (415) 555-0100 & +1-415-555-0100 collapse to one E.164; unparseable → null.
- **Skill aliases** → `reactjs/React.js/REACT` → one `React`; unknowns kept verbatim at lower confidence.

Descoped under time pressure: live LinkedIn scraping (auth-walled / ToS — URL gives the link, exports are parsed); DOCX (PDF/TXT only); ML entity-resolution (rule-based fuzzy match instead). A bare non-US phone with no country code is dropped, not guessed.

Constraints met — **Deterministic & explainable:** pure functions, fixed trust ranks, no randomness, output sorted by id, every field traceable. **Robust:** per-adapter try/except; unknowns → null. **Scale:** O(n) union-find on hashed keys → fine for thousands. Implementation: 6 adapters · 310 passing tests · CLI + minimal web UI.